

How does the architecture work?

A plain-English guide to a memory cell that works quickly while power is on and remembers when power is off.



It is RAM with a built-in memory of its last safe state

UM002B combines two ways of representing the same bit. One is fast but forgetful. The other is slower to change but remembers without power.

The capacitor: the working desk

A capacitor stores a bit as electrical charge. It is quick to read and write, which is why the computer uses it during normal operation.

Charge leaks away. The controller must refresh it while power is on, and it disappears after power is lost.

The memristor: the locked notebook

A memristor stores a bit as a resistance state: an easier or harder path for current through a thin oxide layer.

It does **not** retain charge. Its physical resistance state can remain after power is removed.

The sentence to remember: the capacitor stores charge; the memristor stores resistance. The architecture translates the same digital 0 or 1 between those two physical forms.

What “2T1C1M” means

2T

Two transistors act as separate electronic switches.

+

1C

One capacitor provides the fast, volatile DRAM state.

+

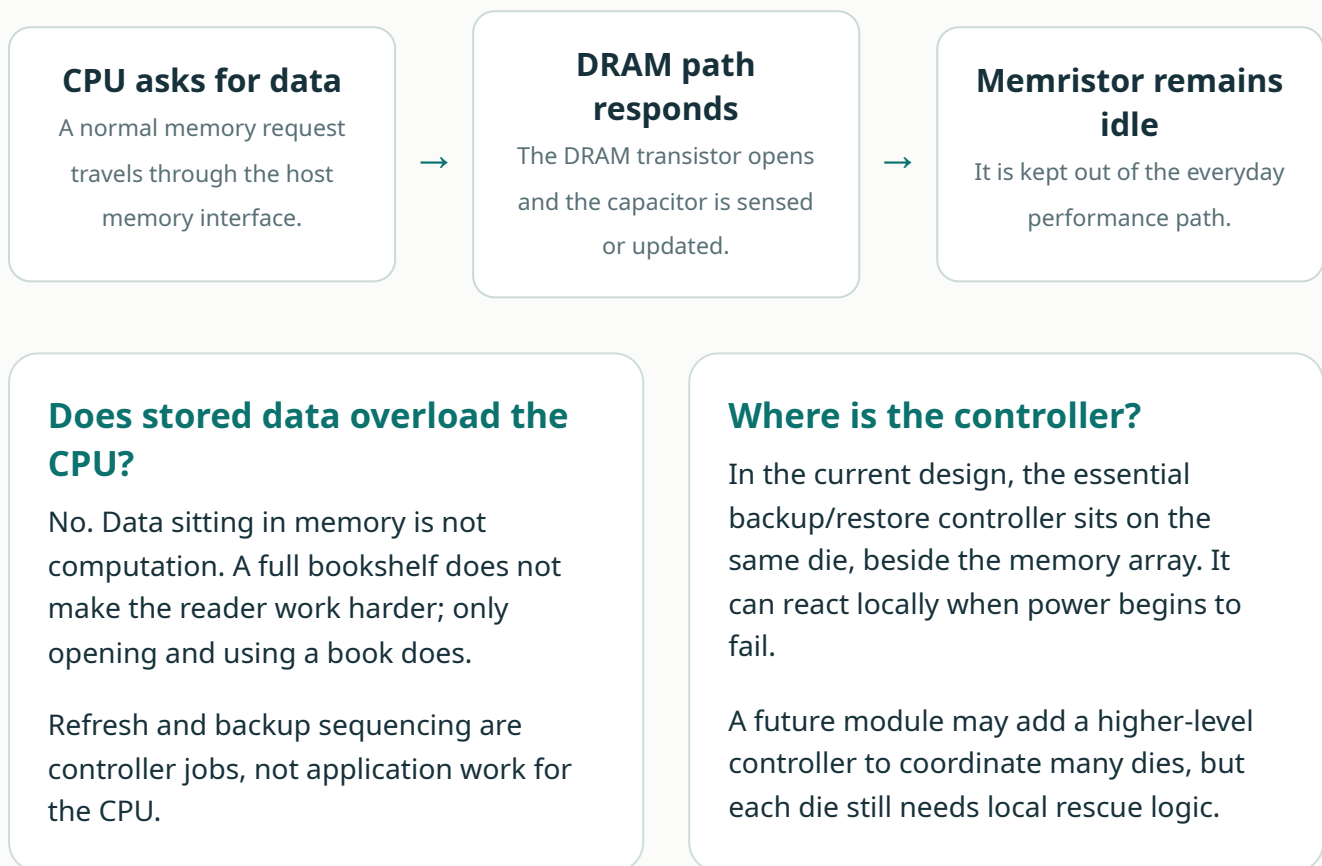
1M

One memristor provides the non-volatile shadow state.

The transistors are not “smart” by themselves. One controls access to the DRAM side; the other controls access to the non-volatile side. The controller decides when each switch should operate.

The computer uses the fast side; the backup side waits

The processor does not continuously read and write the memristor. Ordinary traffic stays on the capacitor side, like conventional dynamic random-access memory (DRAM).



Important: “already in memory” does not mean “already running.” Programs still need CPU time, initialization, decompression, security checks, and sometimes graphics preparation.

The controller tries to create one complete, recoverable snapshot

A power-loss detector warns the on-die controller before the supply disappears completely. Reserve energy must keep the memory alive long enough to finish a controlled backup.

1

Freeze

Stop or drain normal memory traffic so the picture does not keep changing during backup.

2

Translate

Read each capacitor's 0 or 1 and program the matching memristor into a high- or low-resistance state.

3

Verify

Check rows and error-correction information before declaring the new snapshot complete.

4

Commit

Update protected metadata only after the snapshot is complete. This commit marker tells the next boot which version is safe to restore.

5

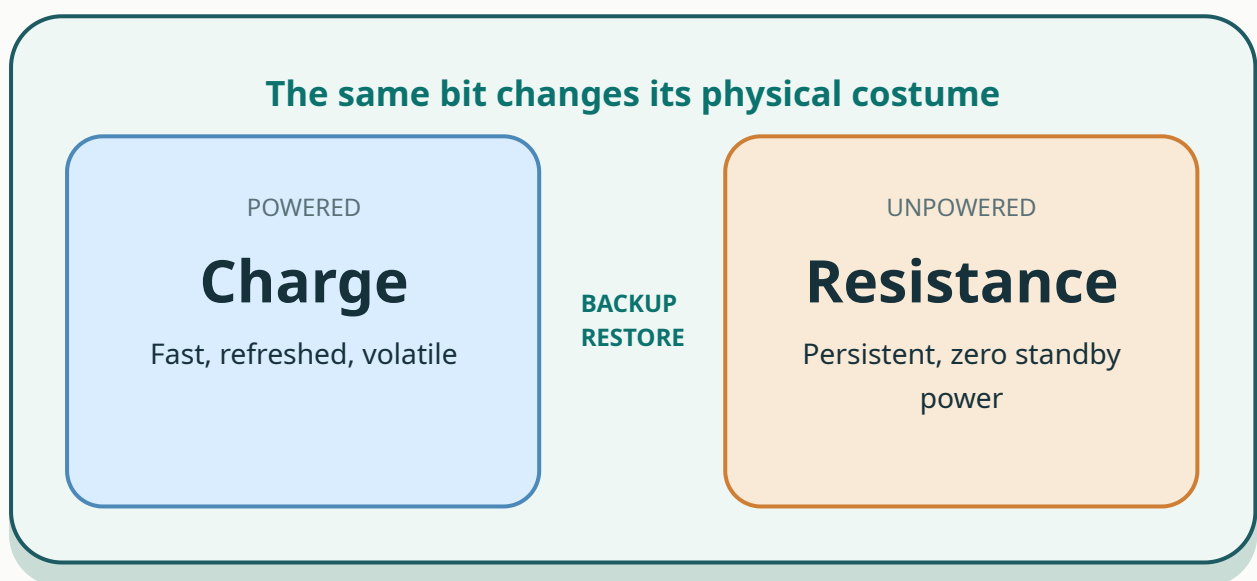
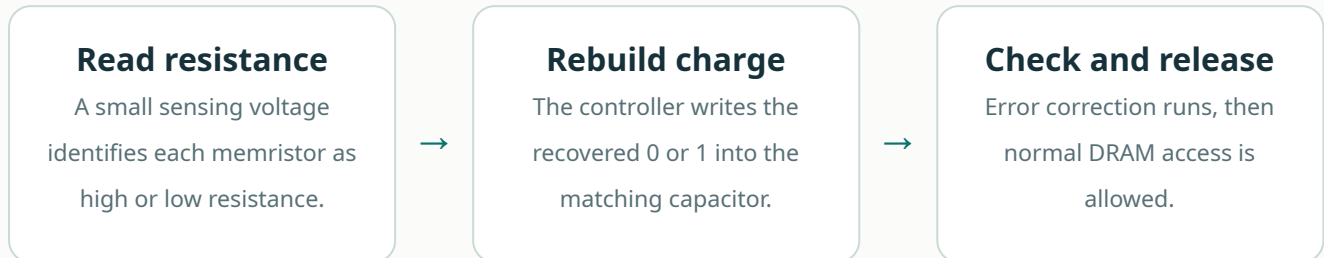
Power down

The capacitors may now discharge. The memristors retain resistance states without standby power.

If power disappears too early: the intended design restores the previous completed snapshot rather than mixing old and partly written rows. The most recent uncommitted changes may be lost, but a torn snapshot should not be presented as valid.

The prototype models include a short backup sequence for a very small demonstration array. A 20 GB, 200 GB, or other product-size device would require a new measured time and energy budget; the prototype timing must not be scaled by assumption.

The last completed snapshot becomes working memory again



What the user experiences

The machine can return to the last successfully captured memory image instead of rebuilding every byte from a separate storage device.

What this does not guarantee yet

"Instant-on" time for a full product has not been measured. Boot firmware, security, device initialization, and operating-system recovery may still add delay.

Core principle: the memristor does not keep charge trapped inside the capacitor. The capacitor is allowed to empty. Later, the retained resistance state tells the controller how to recharge it.

One pool can be shared, but the operating system must manage it

Imagine a 200 GB pool. If files and installed software occupy 100 GB, roughly 100 GB remains available for active work, subject to operating-system overhead and reserved space.



The boundary can move. Delete files and more space becomes available. Create files or install a game and less remains. Occupied bytes do not consume CPU time merely by existing.

When “Save” is pressed

The application and operating system update bytes and metadata in the pool. To make that update durable immediately, the system needs an explicit host-to-controller durability command or a completed snapshot.

A design that backs up only on power failure cannot honestly promise that every application “Save” has already reached the memristor.

When a file is deleted

The operating system marks its space reusable. The old bit pattern may remain physically present until overwritten, just as it can on conventional storage.

Secure deletion therefore needs a defined erase or cryptographic key-destruction policy.

Software matters: a truly unified memory-and-storage pool is not created by the cell alone. The operating system, file system, permissions, crash recovery, and durability rules must all understand the architecture.

Fewer data copies are possible; graphics work does not disappear

Suppose a large game is stored in the unified pool. With suitable operating-system and application support, the CPU can access those bytes without first copying the whole game from a separate SSD into RAM.

Part of the experience	What may improve	What still remains
Launching	Less waiting for bulk storage-to-RAM transfer.	Program startup, authentication, decompression, shader compilation, and engine initialization.
World streaming	Assets can be available from a DRAM-speed system pool.	A discrete graphics card may still copy assets across PCI Express into its own video memory.
Frame rate	Fewer storage stalls may smooth asset delivery.	Rendering speed still depends on the CPU, GPU, memory bandwidth, and software.
Power failure	The last completed memory snapshot may preserve the running state.	Recovery is limited by the snapshot contract; changes not committed before energy runs out can be lost.

The honest performance claim: this architecture aims to remove avoidable movement between storage and working memory. It does not make every application instant, and it does not make the processor or graphics card inherently faster.

How the CPU and GPU react

CPU: ideally sees a familiar memory interface. The on-die controller hides backup and restore. A larger software change is still needed if the same bytes are to behave both as files and as ordinary memory.

GPU: an integrated GPU could potentially share the pool. A discrete GPU normally keeps separate video memory, so one transfer stage can remain.

The architecture is coherent; product behavior still requires silicon

UM002B is presently a pre-silicon architecture supported by analytical models, simulation work, controller design, and prototype-scale planning. It is not yet a manufactured or qualified memory product.

Designed

The operating concept

Separate DRAM and non-volatile access paths, an on-die backup/restore controller, protected snapshot metadata, error correction, and power-fail sequencing.

Modeled

Key electrical and timing behaviors

Device and circuit models explore programming, leakage, thermal effects, disturb, and controller operation. Some modeled corners expose open risks rather than passing.

To prove

The product-level promises

Measured retention, endurance, full-capacity backup energy and time, restore time, yield, manufacturing rules, cost per bit, and real software performance.

Five questions a careful reader should ask

1. How much reserve energy is needed for the intended capacity at the worst temperature and voltage?
2. What is the exact durability rule for an application Save, not just for power failure?
3. How does the design recover when backup stops halfway through?
4. How long do measured resistance states remain readable after years without power?
5. What software changes are required to expose one safe pool for both files and working memory?

Best one-sentence description: UM002B is a fast volatile memory cell paired with a non-volatile shadow and local control logic, designed to preserve a complete recoverable state when power is lost.